

ABC455 F - Merge Slimes 2 题解

题意概括

有一个长度为 N 的数组 A ，初始全为 0 。每次查询给出区间 $[1, r]$ 和 a ：

1. 把区间内每个数都加上 a ；
2. 取出更新后的子数组 $B = A[1..r]$ ，把它看成若干史莱姆的重量；
3. 多次合并史莱姆，合并重量为 x 和 y 的两只史莱姆会产生代价 $x * y$ ；
4. 求最小总代价，对 998244353 取模。

解题思路

第一步：合并顺序其实不重要

设子数组 B 的元素为 $B_1, B_2, \dots, B_M$ 。

把每个史莱姆看作一个“组”，组合并时产生的代价 $x * y$ ，可以理解为：

- 左边组里任选一个元素；
- 右边组里任选一个元素；
- 这样的配对总数就是 $x * y$ 。

从全过程看，每一对最初属于不同组的元素，恰好会在它们第一次被并到同一组时贡献一次代价；而最初就在同一组的元素永远不会贡献。

所以最小总代价其实与合并顺序无关，恒等于：

$$\frac{(\sum B_i)^2 - \sum B_i^2}{2}$$

也就是说，我们只需要知道区间内：

- 元素和 sum
- 元素平方和 sq

答案就是：

$$\frac{sum^2 - sq}{2}$$

第二步：区间加与区间查询

于是题目变成维护数组，支持：

1. 区间加;
2. 查询某个区间的元素和 sum ;
3. 查询某个区间的元素平方和 sq 。

这正适合用懒标记线段树。

若一个区间长度为 len ，所有元素统一加上 v ，那么：

- 新的元素和： $sum + len * v$
- 新的平方和： $sq + 2 * v * sum + len * v^2$

用这个公式更新线段树节点即可。

正确性说明

对固定子数组 B ，总代价等于所有“原本来自不同史莱姆的元素对”数量之和，因此与合并顺序无关。

设：

- $sum = B_1 + B_2 + \dots + B_M$
- $sq = B_1^2 + B_2^2 + \dots + B_M^2$

那么：

$$sum^2 = \sum B_i^2 + 2 \sum_{i < j} B_i B_j$$

移项可得：

$$\sum_{i < j} B_i B_j = \frac{sum^2 - sq}{2}$$

而这正是所有不同初始史莱姆配对贡献的总代价，所以答案公式正确。

线段树部分始终正确维护了每个区间的：

- 区间长度;
- 区间元素和;
- 区间平方和;
- 懒加标记。

因此每次查询都能得到正确的 sum 和 sq ，从而得到正确答案。算法正确。

复杂度

每次查询做一次区间加和一次区间查询。

- 时间复杂度： $O(Q \log N)$
- 空间复杂度： $O(N)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 998244353;
const ll INV2 = (MOD + 1) / 2;

struct Node{
    ll sum;
    ll sq;
    ll lazy;
    int len;
};

vector<Node> seg;

// 由左右儿子回推当前区间的和与平方和
void pull(int id){
    seg[id].sum = (seg[id << 1].sum + seg[id << 1 | 1].sum) % MOD;
    seg[id].sq = (seg[id << 1].sq + seg[id << 1 | 1].sq) % MOD;
}

// 建树时数组初值全为 0，只需要记住区间长度
void build(int id, int l, int r){
    seg[id].sum = 0;
    seg[id].sq = 0;
    seg[id].lazy = 0;
    seg[id].len = r - l + 1;
    if(l == r){
        return;
    }
    int mid = (l + r) >> 1;
    build(id << 1, l, mid);
    build(id << 1 | 1, mid + 1, r);
}

// 整段加 v 时，同时更新区间和、区间平方和与懒标记
void apply(int id, ll v){
```

```

v %= MOD;
ll old_sum = seg[id].sum;
ll len = seg[id].len;

seg[id].sq = (seg[id].sq + 2LL * v % MOD * old_sum % MOD + len % MOD * v %
MOD * v) % MOD;
seg[id].sum = (seg[id].sum + len % MOD * v) % MOD;
seg[id].lazy = (seg[id].lazy + v) % MOD;
}

```

// 下传懒标记, 保证访问子区间前信息正确

```

void push(int id){
    if(seg[id].lazy == 0){
        return;
    }

    apply(id << 1, seg[id].lazy);
    apply(id << 1 | 1, seg[id].lazy);
    seg[id].lazy = 0;
}

```

// 区间加: 把 [ql, qr] 中所有元素都加上 v

```

void update(int id, int l, int r, int ql, int qr, ll v){
    if(ql <= l && r <= qr){
        apply(id, v);
        return;
    }

    push(id);
    int mid = (l + r) >> 1;
    if(ql <= mid){
        update(id << 1, l, mid, ql, qr, v);
    }
    if(qr > mid){
        update(id << 1 | 1, mid + 1, r, ql, qr, v);
    }
    pull(id);
}

```

// 区间查询: 取出 [ql, qr] 的元素和与平方和

```

Node query(int id, int l, int r, int ql, int qr){

```

```

    if(ql <= l && r <= qr){
        return seg[id];
    }

    push(id);
    int mid = (l + r) >> 1;
    if(qr <= mid){
        return query(id << 1, l, mid, ql, qr);
    }
    if(ql > mid){
        return query(id << 1 | 1, mid + 1, r, ql, qr);
    }

    Node left = query(id << 1, l, mid, ql, qr);
    Node right = query(id << 1 | 1, mid + 1, r, ql, qr);

    Node res;
    res.sum = (left.sum + right.sum) % MOD;
    res.sq = (left.sq + right.sq) % MOD;
    res.lazy = 0;
    res.len = left.len + right.len;
    return res;
}

int main(){

    int n, q;
    cin >> n >> q;

    // 线段树维护整个数组 A 的区间和与区间平方和
    seg.assign(4 * n + 5, {0, 0, 0, 0});
    build(1, 1, n);

    while(q--){
        int l, r;
        ll a;
        cin >> l >> r >> a;

        // 先执行题目要求的区间加
        update(1, 1, n, l, r, a);
    }
}

```

```
// 再查询更新后的子数组  $B = A[l..r]$ 
Node res = query(1, 1, n, l, r);

// 最小合并代价恒为  $(sum^2 - sq) / 2$ 
ll ans = (res.sum * res.sum % MOD - res.sq + MOD) % MOD;
ans = ans * INV2 % MOD;
cout << ans << '\n';
}

return 0;
}
```